

Micro Hack 1 - Business Apps Workbook

FY27 - Fabric Apps Motion (EMEA)

Contents

1 Micro Hack 1 — Business Apps Workbook (Participants)	1
1.1 Day agenda (1 day)	1
2 Part A — Understand	2
2.1 1) The scenario in plain words	2
2.2 2) What is Rayfin? (60-second primer)	2
2.3 3) The result you are aiming for	3
3 Part B — Build it step by step	4
3.1 Step 1 — Create and run the empty project	4
3.2 Step 2 — Declare the data model	4
3.3 Step 3 — Build the Bicycle Board (your first screen)	5
3.4 Step 4 — Build the Pit-Stop Queue (take action)	5
3.5 Step 5 — Add the Ride Mood & KPI screen	6
3.6 Step 6 — Add the Live Map (make it shine)	6
3.7 Step 7 — Polish (roles & visuals)	6
4 Wrap-up	7
4.1 Team framing (fill this in 2 minutes)	7
4.2 Demo storyboard (5 minutes)	7
4.3 Reflection	7
4.4 Reference assets	7

1 Micro Hack 1 — Business Apps Workbook (Participants)

Track: **End-customer Apps (Rayfin)** Scenario: **Helios Bicycle** (fictional final customer)

Print-ready PDF: [microhack-1-business-apps-workbook.pdf](#)

This is a **didactic** micro-hack. You don't need prior Rayfin experience — every step explains **what** you do, **why**, and **what you should see**. Follow them in order.

1.1 Day agenda (1 day)

Morning — presentations & demos. Afternoon — hands-on hack.

Time	Session
09:00	Welcome & motion overview — the Fabric apps motion for EMEA, the two paths
09:30	Microsoft Fabric foundations — OneLake, capacity, workspaces
10:00	Demo · Business Apps (Rayfin) — Helios Bicycle Studio
10:30	Break
10:45	Demo · SDC Workloads (Extensibility Toolkit) — GreenGrid Scorecard
11:15	The hack brief, teams & environment check
12:00	Lunch

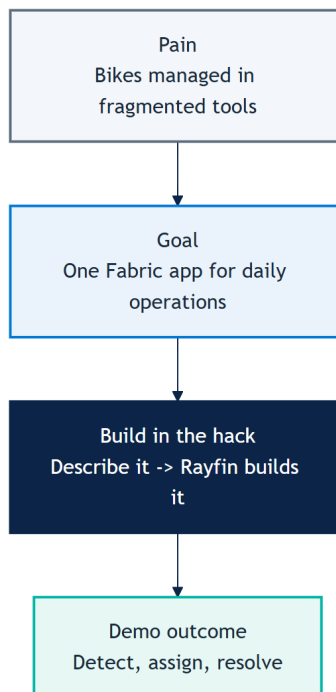
Time	Session
13:30	Hack · Sprint 1 — build the core app (steps 1–4)
15:30	Break
15:45	Hack · Sprint 2 — extend & polish (steps 5–7)
16:45	Team demos (5 min each)
17:00	Wrap-up, KPIs & next steps

The morning shows **both** scenarios (Apps + SDC). This workbook drives the **afternoon hack on the Business Apps track** (Helios Bicycle). The SDC track has its own workbook.

2 Part A — Understand

2.1 1) The scenario in plain words

Helios Bicycle runs shared city bikes across several stations. Today the station teams track bikes, repairs and rider feedback in spreadsheets and chat — slow and error-prone. Your job in the afternoon: build **one simple app**, *Helios Bicycle Studio*, that lets a manager see every bike, fix the right one first, and keep riders happy.



2.2 2) What is Rayfin? (60-second primer)

Rayfin is a *Backend-as-a-Service on Microsoft Fabric*. In plain terms:

- You **describe your data** (e.g. a Bicycle has a code, a station, a status). Rayfin then **creates and manages the database** for you in Fabric — no SQL, no servers to run.
- You **describe your screens in natural language** (with GitHub Copilot). Rayfin generates the app UI against that data.
- So building an app becomes: *declare the data -> describe the screens -> run*.

Key idea to remember: **your data model = your database**. When you declare a Bicycle type, Rayfin makes the matching table automatically.

2.3 3) The result you are aiming for

By the end of the afternoon your app should look like this — a **Bicycle Board** and a **Live Map** of the fleet:

Helios Bicycle Studio
Bicycle Studio

Bicycle Board
Fleet readiness across Helios stations — live operations view

Operations Manager OM

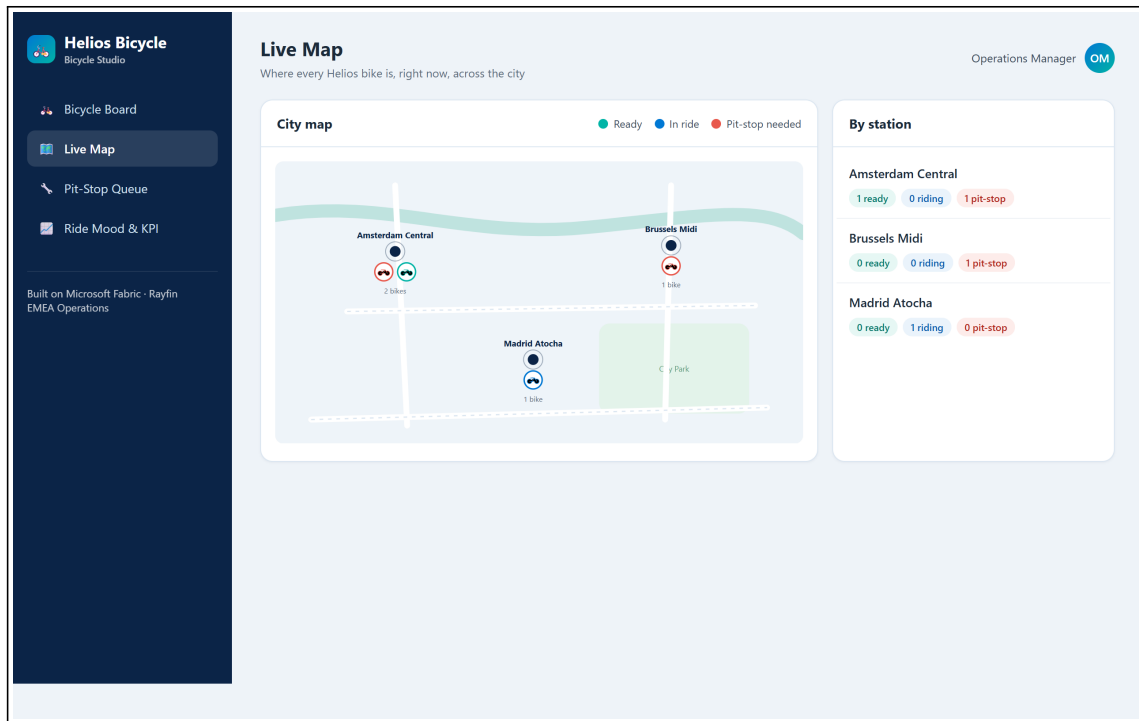
Helios Bicycle Studio
See every bike, fix the right one first, keep riders smiling.

1/4
bikes ready now

Bicycles (4) All stations Amsterdam Central Status: any

BIKE	STATION	STATUS	RIDER MOOD	HEALTH	TAG
HB-AMS-001	Amsterdam Central	Pit-stop needed	★★★★☆ 2.8	25	⚠ Watch
HB-AMS-014	Amsterdam Central	Ready	★★★★★ 4.7	98	★ Star bike
HB-BRU-007	Brussels Midi	Pit-stop needed	★★★★☆ 3.1	40	⚠ Watch
HB-MAD-011	Madrid Atocha	In ride	★★★★★ 4.2	75	Good

Built on Microsoft Fabric · Rayfin EMEA Operations



3 Part B — Build it step by step

How to work: you build the app **conversationally**. You paste a short prompt into **GitHub Copilot** inside the Rayfin project; Copilot writes the code; you check the result in the browser. Each step below = one prompt + what you should see.

3.1 Step 1 — Create and run the empty project

What you do. Scaffold a Rayfin project and start it. “Scaffold” just means *generate a ready-to-run starter project*.

```
npm create @microsoft/rayfin@latest -- --template field-technician
npm run dev # deploys to Fabric and provisions the database
```

Why. `npm run dev` pushes the project to Fabric and **provisions your database** from the data model in `rayfin/data/*.ts`. You now have a live backend before writing a single screen.

**** What you should see.**** A local app opens in the browser with the starter template, and your Fabric workspace shows the new app.

3.2 Step 2 — Declare the data model

What you do. Tell Copilot the data your app is about. This defines the **database tables**.

Build "Helios Bicycle Studio", a fun bike-operations app on Rayfin.
 Data model:
 - Bicycle(bikeCode, station, status[ready|in-ride|pit-stop-needed],
 ↪ featured)
 - RideSession(bicycle, riderAlias, moodScore, startedAt, endedAt)
 - PitStopTicket(ticketCode, bicycle, station, priority[high|normal],
 status[new|assigned|done], issue, openedAt,
 ↪ assignedMechanic)
 - MechanicProfile(displayName, station, active)
 Roles: Operations Manager has full access; Mechanic sees only assigned
 ↪ tickets.
 Style: clean, friendly, Microsoft Fluent, blue #0078d4 + teal #00b4a6.

Why. These four types become four tables. Bicycle.status is a fixed list of values, so the app can colour bikes by status later.

**** What you should see.**** Copilot creates the data files; running the app re-provisions the database with the new tables (still empty screens — that's normal).

3.3 Step 3 — Build the Bicycle Board (your first screen)

What you do. Ask for the main screen: the list of bikes a manager looks at every morning.

Add a "Bicycle Board" screen: a table grouped by station showing a status
 ↪ pill,
 rider mood as stars, a health score and a tag (Star / Good / Watch).
 Add filters for station and status.

Why. This is the app's home base — *detect* which bikes need attention. The health score/tag is computed from status + mood so the manager sees priorities at a glance.

**** What you should see.**** A table of bikes by station with coloured status pills and a tag. You can filter by station and status. (Compare with the first example image above.)

3.4 Step 4 — Build the Pit-Stop Queue (take action)

What you do. Add the screen where a manager turns a problem into an assigned repair.

Add a "Pit-Stop Queue" screen as a kanban with columns new / assigned /
 ↪ done.
 Let me create a ticket from a bike, and assign a mechanic in one click,
 preferring the same-station and least-loaded mechanic.

Why. This is the *assign* -> *resolve* half of the story. "One click" assignment keeps the manager fast; preferring the same-station, least-busy mechanic is a simple, sensible rule.

**** What you should see.**** A three-column board. Creating a ticket from a bike places a card in **New**; assigning moves it to **Assigned** with the mechanic's name.

Milestone (Sprint 1): Steps 1–4 give you a working app — detect a bike, open a ticket, assign it. If you reach here, you can already demo.

3.5 Step 5 — Add the Ride Mood & KPI screen

What you do. Add a light analytics screen.

Add a "Ride Mood & KPI" screen with cards for average rider mood, number of bikes needing a pit-stop, and a workshop -> PoC -> opportunity funnel.

Why. Rider mood is an early quality signal; the funnel cards connect the app to the **business KPI** of the motion (workshops turning into opportunities).

**** What you should see.**** A row of KPI cards with the average mood, the count of bikes needing a pit-stop, and the funnel percentages.

3.6 Step 6 — Add the Live Map (make it shine)

What you do. Add a map view of the fleet.

Add a "Live Map" screen: a stylized city map with one marker per bike colored by status (teal = ready, blue = in ride, red = pit-stop needed), plus a per-station summary panel.

Why. A map turns rows of data into an instantly readable picture — great for the demo and close to how operations teams think about a city fleet.

**** What you should see.**** A map with coloured bike markers and a side panel counting bikes per station (compare with the second example image above).

3.7 Step 7 — Polish (roles & visuals)

What you do. Tighten the experience. Pick what you have time for.

Make the bike rows more visual: add a colored bike illustration whose color reflects the status (teal = ready, blue = in ride, red = pit-stop needed).

Enforce roles: a Mechanic user should only see pit-stop tickets assigned to them.

Why. Visual cues speed up reading; role rules make the app realistic for a real customer rollout.

**** What you should see.**** Coloured bike icons on each row, and a mechanic account that only sees its own tickets.

Milestone (Sprint 2): the app is graphical and role-aware — ready for the 5-minute demo.

4 Wrap-up

4.1 Team framing (fill this in 2 minutes)

- Team name:
- Who is the primary user (manager or mechanic)?
- One sentence: what does your app make faster?

4.2 Demo storyboard (5 minutes)

1. **Context** (30s): who is the user, what hurts today.
2. **Flow** (3m): detect on the Board -> open a ticket -> assign on the Queue -> show the Map.
3. **Architecture** (45s): how the data and the app connect in Fabric (Rayfin provisioned it).
4. **Next step** (45s): what you'd add before a real rollout.

4.3 Reflection

- What was the easiest part of describing the app to Rayfin?
- What did you have to clarify for Copilot to get it right?
- What metric would prove value in week 1 of production?

4.4 Reference assets

- Reference solution code: `src/apprayfin/`
- Rayfin prompt baseline (canonical spec): `src/apprayfin/src/specs/helios-bicycle-app-spec.md`
- Trainer answer key + screenshots: `docs/microhack-1-business-apps-solution.md`